

TP6 – RÉOLUTION DE SAT

Ce sujet étudie la résolution de SAT pour des formules en forme normale conjonctive, en utilisant certaines heuristiques pour accélérer si possible la résolution.

1. STRUCTURES DE DONNÉES

On s'intéresse à des formules logiques en forme normale conjonctive. On note $\mathcal{V}$ l'ensemble des variables.

On représente un littéral en OCaml par :

```
type literal =  
  | X of int  
  | NonX of int
```

OCAML

Les formules logiques en forme normale conjonctive sont représentées par le type :

```
type fnc = literal list list
```

OCAML

Une forme normale conjonctive est donnée par une liste de clauses, et chaque clause par la liste de ses littéraux.

2. LECTURE D'UN FICHIER DIMACS CNF

La plupart des solveurs de SAT peuvent lire en entrée un fichier au format DIMACS CNF, qui est un fichier texte constitué d'une suite de lignes de la forme :

```
p cnf N M  
1 2 -3 0  
-1 3 4 0  
...
```

OCAML

où la première ligne, d'en-tête, contient le nombre de variables N et le nombre de clauses M , et les M lignes suivantes donnent les clauses en forme normale conjonctive. Chaque variable est identifiée par un entier non nul, le littéral opposé par l'opposé du numéro de la variable. Chaque ligne est terminée par 0 .

Question 1. Écrire une fonction `lire_dimacs` qui prend en paramètre un nom de fichier, le lit, et renvoie une valeur de type `fnc` correspondant au contenu du fichier.

Question 2. Ajouter le support des trois extensions suivantes :

- autoriser les lignes vides,
- autoriser la présence de commentaires, uniquement sous la forme de lignes débutant par `#`,
- autoriser l'utilisation de variables nommées, plutôt que numérotées. Ainsi, on pourrait écrire `X_3_4` ou `-X_3_4`.

On pourra en particulier utiliser `String.split_on_char`.

3. SIMPLIFICATION ÉLÉMENTAIRE

Si un littéral apparaît sous la forme x_i et $\overline{x_i}$ dans une clause, alors on peut simplifier la clause en la retirant de la formule. De même, si un littéral apparaît plusieurs fois dans une même clause, on peut simplifier la clause en ne gardant qu'une seule occurrence de ce littéral.

Question 3. Écrire une fonction `simplifie_antagonistes` qui renvoie une forme normale conjonctive après application de ces simplifications.

4. RÈGLE DE PROPAGATION UNITAIRE

Soit F une formule en forme normale conjonctive. On suppose que F contient une clause unitaire réduite au littéral l . Un tel littéral est appelé *littéral isolé*. Nous pouvons alors simplifier la formule F de la manière suivante :

- en supprimant toutes clauses de F qui contiennent l ,
- en supprimant $\neg l$ de toutes les clauses de F (ainsi si une clause ne contient que $\neg l$ elle est remplacée par la clause vide $\perp$),
- on se souvient par ailleurs de la seule valeur possible pour la variable associée.

Par exemple la formule $F = (\neg x_3) \wedge \neg x_3 \vee x_2 \wedge (x_3 \vee \neg x_1)$ contient un unique littéral isolé $\neg x_3$. En simplifiant F par ce littéral, on obtient la formule $F' = \neg x_1$.

On justifie formellement cette simplification. Soit F une formule en forme normale conjonctive contenant un littéral isolé l (associé à la variable propositionnelle p : on a donc $l = p$ ou $l = \neg p$). F peut s'écrire alors sous la forme $F = l \wedge F_1 \wedge F_2 \wedge F_3$, où :

- F_1 est une formule constituée de clauses contenant chacune le littéral l ;
- F_2 est une formule constituée de clauses contenant chacune le littéral $\neg l$ et ne contenant pas le littéral l ;
- F_3 est une formule constituée de clauses ne contenant chacune ni le littéral l , ni le littéral $\neg l$.

Remarquons que chacune des formules F_1 , F_2 et F_3 peut être réduite à la formule vide, satisfiable.

La formule simplifiée de F par le littéral l est alors la formule $F' = F'_2 \wedge F_3$, où F'_2 s'obtient à partir de F_2 en supprimant toutes les occurrences du littéral $\neg l$ dans chacune des clauses de F_2 .

Question 4. Soit σ une valuation de $\mathcal{V} \setminus \{p\}$. Montrer que σ satisfait F' si et seulement s'il existe une valuation $\bar{\sigma}$ de $\mathcal{V}$ prenant les mêmes valeurs que σ sur $\mathcal{V} \setminus \{p\}$ qui satisfait F . On précisera la valeur de $\bar{\sigma}(p)$.

Question 5. Appliquer l'algorithme de propagation unitaire à la formule

$$F = x_1 \wedge (x_4 \vee x_6 \vee x_7) \wedge (\neg x_1 \vee \neg x_6)$$

Question 6. Écrire une fonction `nouveau_lit_isole` qui, à partir d'une formule F , renvoie un littéral isolé de F . Si F ne contient pas de tel littéral, la fonction renverra le littéral x_0 qui n'est pas utilisé comme variable propositionnelle par ailleurs.

Question 7. Écrire une fonction `simplification` qui, à partir d'un littéral l et d'une formule F , renvoie la formule simplifiée F' après propagation du littéral l dans F .

Question 8. Écrire une fonction `propagation` qui effectue cette simplification sur une formule, tant qu'elle est possible.

Question 9. On appelle *taille* d'une formule F le nombre total de littéraux apparaissant dans ses clauses. Évaluer le nombre d'opérations effectuées par les fonctions `nouveau_lit_isole`, `simplification`, puis par la fonction `propagation` quand elles s'appliquent à une formule de taille n .

5. RÈGLE DU LITTÉRAL INFRUCTUEUX

On décrit maintenant une autre méthode de déduction plus puissante combinant la propagation unitaire et une opération appelée *règle du littéral infructueux* décrite ci-dessous.

Étant donné une formule F en forme normale conjonctive et une variable propositionnelle x :

- si l'algorithme de propagation unitaire appliqué à la formule $F \wedge \neg x$ permet de déduire la clause vide alors on ajoute la clause x à F ;
- si l'algorithme de propagation unitaire appliqué à la formule $F \wedge x$ permet de déduire la clause vide alors on ajoute la clause $\neg x$ à F .

Question 10. Justifier formellement que si l'on peut déduire la clause vide à partir de la formule $F \wedge \neg x$ alors $F \equiv F \wedge x$.

Question 11. Écrire une fonction `variables` qui, à partir d'une formule, renvoie la liste de ses variables sans doublons.

On rappelle que la fonction `List.flatten` permet de concaténer tous les éléments d'une liste de listes.

Question 12. Écrire une fonction `deduction` qui, à partir d'un tableau, d'une variable x et d'une formule F , renvoie 1 si la règle du littéral infructueux permet d'ajouter la clause x à F , -1 si elle permet d'ajouter la clause $\neg x$ à F et 0 sinon.

Nous proposons un deuxième algorithme de propagation basé sur la règle du littéral infructueux. Celui-ci consiste à appliquer la propagation unitaire et, quand celle-ci ne permet plus de déduire de nouvelles clauses, à appliquer la règle du littéral infructueux pour obtenir une nouvelle clause unitaire de la forme x ou $\neg x$. Dès lors on peut reprendre la propagation unitaire. Le processus s'arrête lorsque ni la propagation unitaire ni la règle du littéral infructueux ne permettent de déduire de nouvelles clauses.

Question 13. Écrire une fonction `propagation2` qui met en œuvre cet algorithme.

6. RÉSOUDRE SAT

Question 14. Écrire une fonction qui résout SAT pour les formes normales conjonctives, la plus efficace que vous pourrez.

7. TRANSITION DE PHASE

Dans cette partie, on fixe arbitrairement un nombre N de variables propositionnelles. On s'intéresse à une restriction du problème de la satisfiabilité, que l'on note $n, p - SAT$, où n et p sont des entiers.

On ne considère que la satisfiabilité de formules de la forme :

$$\bigwedge_{i=1}^n \bigvee_{j=1}^p a_{i,j}$$

où les $a_{i,j}$ sont des littéraux.

On cherche à estimer la probabilité qu'une telle formule soit satisfiable en fonction du paramètre $\rho = n/p$. On va donc générer au hasard des formules et compter la proportion de formules satisfiables parmi elles.

On utilisera pour cela en OCaml les fonctions du module `Random` :

```
Random.init : int -> unit
Random.int : int -> int
```

OCAML

La fonction `Random.init` sert à initialiser la graine du générateur de nombres aléatoires (une même graine génèrera la même suite de nombre aléatoires : dans des applications réelles, il est courant d'initialiser cette graine avec une donnée qui varie, par exemple la date ou des données provenant du pool d'entropie de l'ordinateur, telles que l'activité réseau, clavier, souris, etc., récente). Cette fonction doit être appelée une seule fois avant de générer le moindre nombre aléatoire.

Par la suite, chaque appel à `Random.int k` renvoie un entier tiré uniformément dans $\llbracket 0, k - 1 \rrbracket$.

Question 15. Écrire une fonction `random_litteral : int → litteral` qui prend en paramètre un entier N et choisit au hasard uniformément un littéral parmi $x_0, x_1, \dots, x_{N-1}, \neg x_0, \dots, \neg x_{N-1}$.

Question 16. Écrire une fonction `random_clause : int → int → litteral list` qui prend en argument deux entiers N et p et construit au hasard uniformément une clause de la forme $\bigvee_{j=1}^p a_j$, où les a_j sont des littéraux pris parmi N variables.

Question 17. Écrire une fonction `random_instance : int → int → int → fnc` qui prend en arguments trois entiers N , n et p et construit une instance de $n, p - SAT$ tirée au hasard avec au plus N variables.

Question 18. Écrire une fonction `proportion_npsat : int → int → int → float` qui prend en arguments trois entiers N , n et p , qui génère un nombre raisonnable de votre choix d'instances de $n, p - SAT$ et qui renvoie la proportion d'instances

| satisfiable.

| **Question 19.** Écrire une fonction `proportion_sat : int → float → float` qui prend en arguments un entier N et un flottant ρ et qui estime la probabilité qu'une formule de $n, p - SAT$ soit satisfiable lorsque les entiers n et p vérifient $\rho = n/p$.

| **Question 20.** Faites varier ρ pour N fixé. Constatez-vous quelque chose ?

Last updated 2025-11-17 11:25:20 +0100